

Hướng dẫn tham gia

QuantVN Oracle Challenge

World Cup 2026

Dự đoán từng trận, đội vô địch, và chọn ngựa ô

1. Giới thiệu

Challenge này tổ chức trong khuôn khổ World Cup 2026, diễn ra từ ngày 12/6 đến 20/7/2026 với 48 đội tham dự và 104 trận đấu. Ý tưởng rất đơn giản: bạn viết một đoạn code Python để dự đoán kết quả các trận đấu, chọn đội nào sẽ vô địch, và đặt cược vào một đội “ngựa ô” mà bạn cho rằng sẽ đi xa hơn kỳ vọng. Chúng tôi chấm điểm dựa trên độ chính xác theo công thức Brier Score, con số càng thấp thì dự đoán càng tốt.

Không cần biết machine learning. Không cần là dân quant hay phân tích dữ liệu chuyên nghiệp. Chỉ cần biết Python cơ bản và có chút hiểu biết về bóng đá là đủ để tham gia. Nhiều người chơi ở Level 1 chỉ bằng cách chỉnh vài con số theo cảm nhận bóng đá của mình mà vẫn có kết quả khá tốt.

2. Ba việc bạn cần làm

Trong file submission của mình, bạn cần hoàn thiện ba phần. Cả ba nằm trong một file Python duy nhất, và bạn có thể gửi lại bản cập nhật bất cứ lúc nào trước trận tiếp theo.

Phần 1: Dự đoán kết quả từng trận

Hàm `predict()` sẽ được gọi tự động trước mỗi trận. Bạn nhận vào thông tin hai đội, dữ liệu từ thị trường Polymarket, và trả về xác suất ba kết quả: đội A thắng, hòa, đội B thắng. Tổng ba con số phải bằng đúng 1.0 (tương đương 100%).

Deadline cho phần này là trước giờ bóng lăn của từng trận. Organizer ghi nhận thời điểm nhận được file của bạn để xác định trận nào được tính điểm.

Phần 2: Dự đoán đội vô địch

Hàm `predict_champion()` trả về xác suất vô địch của cả 48 đội. Đây là dự đoán một lần duy nhất, được khóa trước khi vòng bảng kết thúc. Sau khi qua deadline, bạn không thể thay đổi dự đoán này nữa cho dù sau đó có trận nào làm bạn thay đổi suy nghĩ.

Deadline: 28/6/2026, 23:59 giờ Việt Nam.

Phần 3: Chọn ngựa ô

Biến DARK_HORSE là tên một đội bạn cho rằng sẽ đi xa hơn kỳ vọng của thị trường. Kiểu như: “đội này yếu trên giấy nhưng tôi linh cảm sẽ vào tứ kết”. Nếu đúng, bạn được xét Chaos Award. Cùng deadline với phần 2.

Nếu đội bạn chọn có xác suất vô địch trên Polymarket dưới 5% lúc lock mà sau đó vào bán kết trở lên, bạn còn được cộng thêm điểm thưởng.

3. Dữ liệu bạn có trong tay

Khi hàm predict() được gọi, bạn nhận được ba nguồn thông tin.

Thông tin trận đấu

Bao gồm tên hai đội, vòng đấu (vòng bảng hay knockout), nhóm bảng, giờ thi đấu theo giờ Việt Nam, và kết quả hiện tại (luôn là null khi bạn đang dự đoán, vì đây là trận chưa đá).

Thông tin 48 đội

File teams.json chứa thông tin từng đội: hạng FIFA hiện tại, chỉ số Elo (đo sức mạnh tổng hợp dựa trên lịch sử đối đầu qua nhiều năm), và liên đoàn châu lục. Chỉ số Elo là công cụ chính cho những bạn không muốn phụ thuộc vào kết nối internet.

Dữ liệu live từ Polymarket

Polymarket là sàn dự đoán lớn nhất thế giới, nơi người ta đặt tiền thật vào xác suất kết quả. Dữ liệu trả về bao gồm xác suất vô địch toàn giải của từng đội, volume giao dịch, và thanh khoản.

Lưu ý quan trọng: Polymarket chỉ cung cấp odds vô địch toàn giải, không phải odds của từng trận cụ thể. Bạn dùng nó như một proxy đo sức mạnh tương đối giữa hai đội đang chạm trán trong trận đó. Đội nào được thị trường đánh giá cao hơn thì có khả năng thắng trận đó cao hơn.

4. Ba cách tiếp cận theo trình độ

Cách 1: Chỉ dùng chỉ số Elo

Đây là cách đơn giản nhất và không cần kết nối internet. Chỉ số Elo là hệ thống xếp hạng dựa trên kết quả các trận đấu trong quá khứ. Đội càng đánh bại nhiều đối thủ mạnh thì Elo càng cao. FIFA, cờ vua, và nhiều giải thể thao khác đều dùng biến thể của hệ thống này.

Xác suất đội A thắng được tính bằng công thức Elo chuẩn: chia 1 cho $(1 + 10^{\frac{Elo_B - Elo_A}{400}})$. Phần hòa bạn ước lượng bằng một hàm giảm dần theo khoảng cách Elo — hai đội càng cân sức thì tỉ lệ hòa càng cao, khoảng 27% khi Elo bằng nhau và giảm nhanh khi chênh lệch lớn.

Ưu điểm: đơn giản, ổn định, không phụ thuộc internet. Nhược điểm: không cập nhật theo phong độ gần đây, không biết đội nào đang bị chấn thương.

```
def predict(match, teams, market_data):
    elo_a = teams.get(match['team_a'], {}).get('elo', 1500)
    elo_b = teams.get(match['team_b'], {}).get('elo', 1500)
    win_a = 1 / (1 + 10 ** ((elo_b - elo_a) / 400))
    win_b = 1 - win_a
    draw = 0.27 * math.exp(-abs(elo_a - elo_b) / 600)
    raw = {'team_a': win_a*(1-draw), 'draw': draw, 'team_b': win_b*(1-
draw)}
    if match['stage'] != 'Group Stage':
        raw['draw'] = 0.0
    return normalize(raw)
```

Cách 2: Trộn Polymarket và Elo

Polymarket phản ánh kỳ vọng của thị trường tại thời điểm hiện tại, có thể đã tính đến chấn thương, thẻ đỏ, phong độ gần đây mà Elo không biết. Còn Elo phản ánh sức mạnh lịch sử ổn định hơn. Kết hợp hai nguồn này thường cho kết quả tốt hơn chỉ dùng một mình.

Công thức đơn giản: lấy 60% từ tỉ lệ Polymarket và 40% từ Elo. Nếu đội nào không có trên Polymarket thì dùng 100% Elo. Tỉ lệ 60-40 này là điểm khởi đầu hợp lý, bạn có thể chỉnh theo cảm nhận của mình.

```
def predict(match, teams, market_data):
    pm = market_data['polymarket_probability']
    pm_a = pm.get(match['team_a'], 0.0)
    pm_b = pm.get(match['team_b'], 0.0)
    elo_a = teams.get(match['team_a'], {}).get('elo', 1500)
    elo_b = teams.get(match['team_b'], {}).get('elo', 1500)
    e_a = math.exp(elo_a / 400)
    e_b = math.exp(elo_b / 400)
    elo_wa = e_a / (e_a + e_b)
    if pm_a + pm_b > 0:
        pm_wa = pm_a / (pm_a + pm_b)
        win_a = 0.6 * pm_wa + 0.4 * elo_wa
    else:
        win_a = elo_wa
    win_b = 1 - win_a
    draw = 0.27 * math.exp(-abs(elo_a - elo_b) / 700)
    raw = {'team_a': win_a*(1-draw), 'draw': draw, 'team_b': win_b*(1-
draw)}
    if match['stage'] != 'Group Stage':
        raw['draw'] = 0.0
    return normalize(raw)
```

Cách 3: Nâng cao hơn

Nếu muốn đào sâu hơn, bạn có thể điều chỉnh trọng số Polymarket theo volume giao dịch. Đội nào được giao dịch nhiều hơn thì số liệu Polymarket của họ đáng tin hơn — thị trường đã “suy

ngiht kỹ” về đội đó. Bạn cũng có thể thêm một hệ số nhỏ cho các đội CONCACAF (Mỹ, Canada, Mexico) vì giải đấu lần này tổ chức trên sân nhà của họ.

Đây là hướng mở rộng để bạn tự khám phá. Không có giới hạn nào cho Level 3 — bạn có thể thử Dixon-Coles, mô hình Poisson, hay bất kỳ gì bạn muốn.

5. Cách tính điểm — Brier Score

Brier Score đo độ chính xác của dự đoán xác suất, không phải dự đoán kết quả nhị phân. Nghĩa là bạn không chỉ cần đoán đúng ai thắng, mà còn cần đặt con số xác suất hợp lý. Dự đoán “Mexico thắng với xác suất 99%” sẽ bị phạt nặng nếu Mexico thua, vì quá tự tin.

Công thức

$$BS = (\text{xác suất đặt cho đội A} - \text{kết quả thực tế đội A})^2 + (\text{xác suất đặt cho hòa} - \text{kết quả thực tế hòa})^2 + (\text{xác suất đặt cho đội B} - \text{kết quả thực tế đội B})^2$$

Kết quả thực tế là 1 nếu sự kiện xảy ra, 0 nếu không. Con số cuối cùng là trung bình của tất cả các trận.

Ví dụ cụ thể

Trận Mexico vs Nam Phi, kết quả: Mexico thắng.

```
Bạn đặt: {'team_a': 0.60, 'draw': 0.20, 'team_b': 0.20}
BS = (0.60-1)^2 + (0.20-0)^2 + (0.20-0)^2
    = 0.16 + 0.04 + 0.04 = 0.24    <-- tốt

Nếu đặt đều 1/3:
BS = (0.33-1)^2 + (0.33-0)^2 + (0.33-0)^2 = 0.667    <-- baseline
```

Mục tiêu là đưa Brier Score xuống dưới 0.667. Mô hình tốt thường đạt khoảng 0.3 đến 0.5 cho các giải bóng đá lớn.

Lưu ý vòng knockout

Ở vòng knockout không bao giờ có kết quả hòa vì ai thua sẽ bị loại sau hiệp phụ hay sút luân lưu. Bạn bắt buộc phải set xác suất hòa = 0 ở các vòng này. Nếu bạn để xác suất hòa khác 0, phần đó sẽ bị phạt toàn bộ vì kết quả thực tế lúc nào cũng là 0 mà bạn lại đặt số khác 0.

6. Quy trình nộp bài

Bước 1: Tải repo về máy. Bấm nút “Code” màu xanh trên GitHub rồi chọn “Download ZIP”, giải nén ra Desktop.

Bước 2: Cài thư viện cần thiết bằng lệnh sau trong cửa sổ lệnh (Windows: Command Prompt, Mac: Terminal):

```
pip install -r requirements.txt
```

Bước 3: Mở file `predict_template.py`, viết logic vào ba phần đã đánh dấu. Chạy thử để kiểm tra:

```
python predict_template.py
```

Bước 4: Đổi tên file thành tên của bạn và đặt vào thư mục `submissions/`:

```
cp predict_template.py submissions/nguyen-van-a.py # Mac
copy predict_template.py submissions\nguyen-van-a.py # Windows
```

Bước 5: Kiểm tra bài hợp lệ:

```
python evaluator.py --test submissions/nguyen-van-a.py
```

Bước 6: Gửi file cho organizer qua email, hoặc upload lên GitHub cá nhân rồi gửi link.

Về deadline: phần dự đoán từng trận được tính theo thời điểm organizer nhận được file. Bạn cần gửi trước giờ bóng lăn. Phần vô địch và ngựa ô được lock cứng lúc 23:59 ngày 28/6/2026 — sau thời điểm này không thể thay đổi dự đoán cho dù giải vẫn còn đang diễn ra.

Bạn được phép cập nhật model trong suốt giải. Gửi lại file mới cho organizer trước trận tiếp theo là được. Per-match tính theo bản nhận gần nhất trước trận. Vô địch và ngựa ô thì không thay đổi được sau deadline.

7. Một vài câu hỏi thường gặp

Polymarket không fetch được thì sao?

Không sao. Model vẫn chạy bình thường dựa hoàn toàn vào Elo. `market_data['market_found']` sẽ trả về False và mọi `pm.get(t, 0.0)` sẽ bằng 0.0. Đây cũng là lý do bạn nên có logic fallback trong code — như trong các ví dụ ở trên, nếu `pm_a + pm_b = 0` thì dùng 100% Elo.

Baseline là bao nhiêu?

Đặt đều 1/3 cho mỗi kết quả thì Brier Score khoảng 0.667 mỗi trận. Đây là baseline — bất kỳ model nào tốt hơn “đoán mò” đều phải thấp hơn con số này.

Phần vô địch được chấm thế nào trong lúc giải đang diễn ra?

Sau mỗi vòng đấu, organizer cập nhật snapshot interim: các đội đã bị loại có outcome = 0 và phần Brier đó được chốt lại. Đây là điểm tham khảo, chưa phải điểm cuối. Điểm chính thức sẽ được tính sau Chung kết ngày 20/7 khi biết chính xác đội vô địch.

Tôi không quen Python, có tham gia được không?

Được. Chỉ cần copy phần template Level 1, điều chỉnh vài con số theo cảm tính bóng đá của mình là đủ. Ví dụ bạn thích Brazil, bạn có thể thêm vào sau khi tính Elo: `raw['team_a'] *= 1.15` nếu team_a là Brazil. Đơn giản vậy thôi. File `predict_template.py` có sẵn ví dụ và ghi chú chi tiết để bạn copy.

Tôi có thể sử dụng thư viện ngoài không?

Được, trong danh sách có sẵn: numpy, pandas, scipy, scikit-learn, xgboost, lightgbm. Không sử dụng model pretrained nặng (PyTorch, TensorFlow). Các thư viện này đã có trong `requirements.txt` nên chỉ cần pip install một lần.